

Repair Shop Application Deployment on AWS

Overview of Project

Scenario:

RepairHub, a local repair services company with 50 technicians across the city, faces service management challenges due to distributed field operations and multiple repair locations.

Solution:

A cloud-based Repair Management Application hosted on AWS. The architecture features a Next.js frontend on AWS Amplify, Node.js backend on Elastic Beanstalk, and CloudFront for secure content delivery. This scalable application centralizes repair tracking, enables real-time service coordination, and improves workflow efficiency across all repair locations and field teams.

About Project:

In this project, I will intentionally introduced some errors while building the application, and then walked through the troubleshooting process using services like Amazon CloudWatch. By the end, I gained hands-on experience in both application development and diagnosing real-world issues, just like a cloud support engineer!

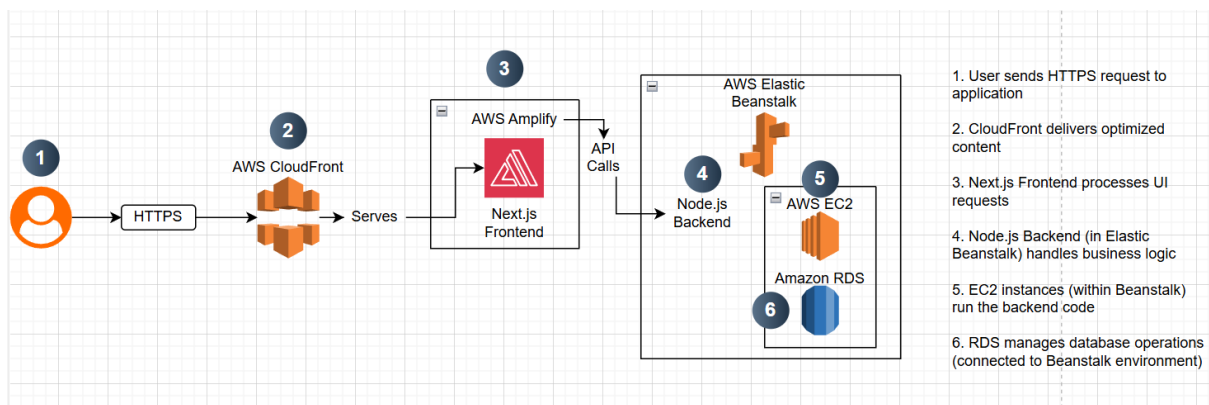
Steps performed:

1. Deployed the Backend on AWS Elastic Beanstalk
2. Deployed the Frontend on AWS Amplify
3. Connected the Frontend and Backend
4. Implemented AWS CloudFront for secure content delivery

Services Used:

- **AWS Amplify:** Hosting service for the frontend application that provides continuous deployment capabilities.
- **AWS Elastic Beanstalk:** Managed service for deploying and scaling the backend application.
- **Amazon RDS:** Managed relational database service for storing application data.
- **AWS CloudFront:** Content delivery network that provides secure HTTPS connections.
- **AWS IAM:** Identity and Access Management service for controlling access to AWS resources.

This is the architectural diagram for the project:



Deploying the Backend on AWS Elastic Beanstalk

Objective: Set up a Managed Backend Environment for Troubleshooting Practice

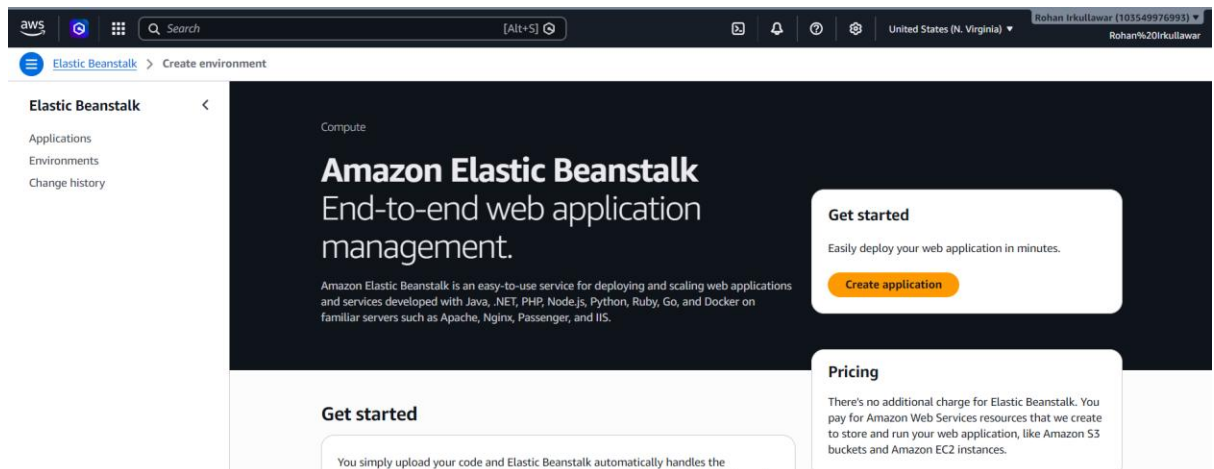
I needed to deploy a Node.js backend for my Todo application using a managed service that would allow me to practice cloud support engineering skills, including troubleshooting and diagnostics.

Solution: Deploying with AWS Elastic Beanstalk

AWS Elastic Beanstalk provides a managed platform that automatically handles infrastructure provisioning, application deployment, capacity management, and health monitoring. For my Todo application, this meant I could focus on developing features rather than managing infrastructure.

Configured Environment

1. I logged in to my AWS management console and navigated to Elastic Beanstalk from the search bar.
2. Clicked Create Application to begin setup.



3. For the Environment tier, I selected Web server environment. This tier is optimized for HTTP-based applications that serve requests over the internet, which matches my REST API backend requirements.
4. I entered backend-repairs as the application name, and the environment name auto-populated.

Application information [Info](#)
Application name

Maximum length of 100 characters.

► **Application tags (optional)**

Environment information [Info](#)
Choose the name, subdomain and description for your environment. These cannot be changed later.
Environment name

Must be from 4 to 40 characters in length. The name can contain only letters, numbers, and hyphens. It can't start or end with a hyphen. This name must be unique within a region in your account.

5. For Domain Name, I entered backend-repairs as my domain name in ElasticBeanstalk. By default, AWS ElasticBeanstalk assigns the application a subdomain under elasticbeanstalk.com (e.g. backend-repairs.region.elasticbeanstalk.com).

Environment information [Info](#)
Choose the name, subdomain and description for your environment. These cannot be changed later.
Environment name

Must be from 4 to 40 characters in length. The name can contain only letters, numbers, and hyphens. It can't start or end with a hyphen. This name must be unique within a region in your account.

Domain
 .us-east-1.elasticbeanstalk.com [Check availability](#)

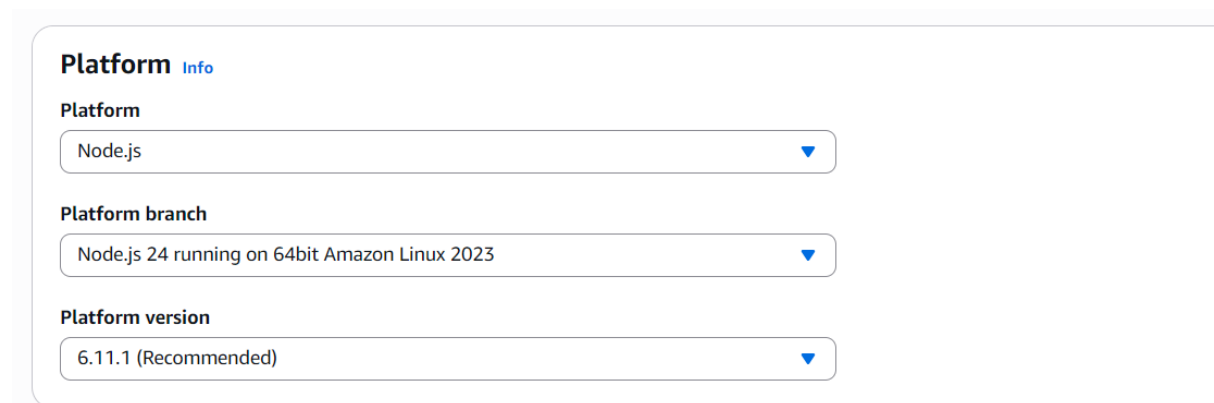
► **Environment description**

Key Information:

- ElasticBeanstalk automatically creates a CNAME that points to the environment's load balancer
- The application URL can be viewed in the ElasticBeanstalk console environment overview
- If a custom domain is needed, it can later be configured using Amazon Route 53

Selected Platform

1. Under Platform, I selected:
 - Platform: Node.js
 - Platform Branch: Node.js 22 running on 64-bit Amazon Linux 2023. Node.js 18 is a Long-Term Support (LTS) version, providing stability, security updates, and compatibility with modern JavaScript features.
 - Platform Version: Left as default.



The screenshot shows the 'Platform' configuration section in the AWS Elastic Beanstalk console. It includes three dropdown menus: 'Platform' set to 'Node.js', 'Platform branch' set to 'Node.js 24 running on 64bit Amazon Linux 2023', and 'Platform version' set to '6.11.1 (Recommended)'. Each dropdown has a blue downward arrow on the right side.

Configuration Item	Selected Value
Platform	Node.js
Platform branch	Node.js 24 running on 64bit Amazon Linux 2023
Platform version	6.11.1 (Recommended)

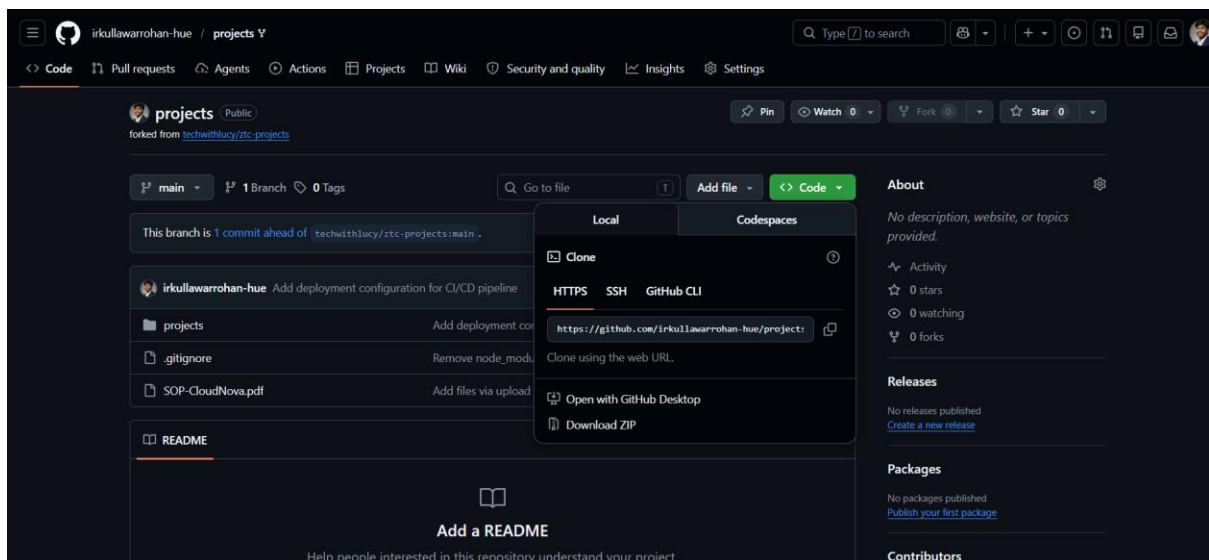
Uploaded Application Code

1. Under Application code, I selected:

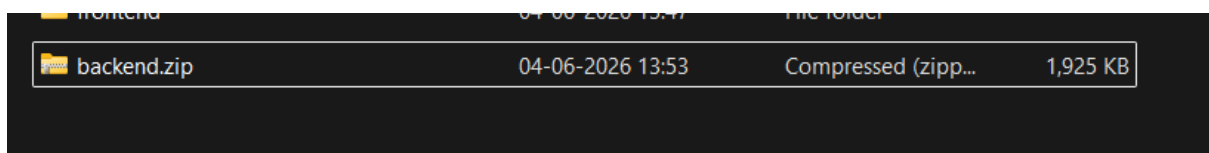
- Upload your code
- Version label: V1
- Downloaded the project repository from GitHub:

Repository: <https://github.com/irkullawarrohan-hue/projects>

- I clicked code -> Download ZIP
- Extracted ZIP file locally
- Navigated to the backend project directory



- I created a ZIP archive of the backend folder only.



Application code [Info](#)

- ☐ Sample application
- ☐ Existing version
Application versions that you have uploaded.
- ☒ Upload your code
Upload a source bundle from your computer or copy one from Amazon S3.

Version label

Unique name for this version of your application code.

V1

Source code origin. Maximum size 500 MB

- ☒ Local file

Upload application

 Choose file

✓ File name: **backend.zip**

File must be less than 500MB max file size

- ☐ Public S3 URL

Presets [Info](#)

Start from a preset that matches your use case or choose custom configuration to unset recommended values and use the service's default values.

Configuration presets

- ☒ Single instance (free tier eligible)
- ☐ Single instance (using spot instance)
- ☐ High availability
- ☐ High availability (using spot and on-demand instances)
- ☐ Custom configuration

[Cancel](#)

[Next](#)

Step 4: Configured Service Access

- In the Service access section, I selected Create and use new service role. Creating a dedicated role with specific permissions follows AWS security best practices by limiting access to only what's needed.
- Clicked on Create role for both fields and got the roles created.
- The service role name auto-populated as aws-elasticbeanstalk-service-role.
- The instance profile auto-populated as aws-elasticbeanstalk-ec2-role.

Configure service access [Info](#)

Service access
IAM roles, assumed by Elastic Beanstalk as a service role, and EC2 instance profiles allow Elastic Beanstalk to create and manage your environment. Both the IAM role and instance profile must be attached to IAM managed policies that contain the required permissions. [Learn more](#)

Service role
Choose an IAM role for Elastic Beanstalk to assume as a service role. The IAM role must have the required IAM managed policies.

aws-elasticbeanstalk-service-role ▼

↻

Create role ↗

EC2 instance profile
Choose an IAM instance profile with managed policies that allow your EC2 instances to perform required operations.

aws-elasticbeanstalk-ec2-role ▼

↻

Create role ↗

EC2 key pair - optional
Select an EC2 key pair to securely log in to your EC2 instances. [Learn more](#)

Choose a key pair ▼

↻

Cancel

Skip to Review

Previous

Next

Database Configuration

Objective: Set up a Reliable Database and Configure Monitoring

I set up a managed MySQL database using Amazon RDS and configured essential monitoring and logging features. These steps ensured my application had a reliable data store with proper operational visibility.

The VPC and Instance Settings remained at their default configurations, which were pre-populated and suitable for the initial deployment.

Step 1: Configure Database Engine

- In the Database section, I enabled the database option. Using RDS instead of a local database provides better reliability, automated backups, and simplified scaling.

Database [Info](#)

Integrate an RDS SQL database with your environment. [Learn more](#) ↗

☒ Enable database

- Engine: MySQL
- Engine version: 8.0.x
- Instance class: db.t3.micro (Free tier eligible)
- Storage: 5 GB
- I set a username and password and stored these securely.

Database settings

Choose an engine and instance type for your environment's database.

Engine

mysql ▼

Engine version

8.4.8 ▼

Instance class

db.t3.small ▼

Storage

Choose a number between 5 GB and 1024 GB.

20

GB

Username

TodoA

Password

.....

I skipped the Configure instance traffic and scaling section.

Configure Monitoring and Logging

- In the Monitoring section, I selected Enhanced Health Reporting to enable real-time monitoring of the application and EC2 instances.

▼ Monitoring [Info](#)

Health reporting

Enhanced health reporting provides free real-time application and operating system monitoring of the instances and other resources in your environment. The **EnvironmentHealth** custom metric is provided free with enhanced health reporting. Additional charges apply for each custom metric. For more information, see [Amazon CloudWatch Pricing](#).

System

- ☐ Basic
- ☒ Enhanced

- Under Health event streaming to CloudWatch Logs, I activated Log Streaming by selecting the Activated checkbox.
- Set Retention to 1 day to minimize potential CloudWatch Logs charges.
- Under Lifecycle, I checked Delete logs upon environment termination. This ensures that logs are automatically removed when the environment is deleted, helping to avoid unnecessary storage costs.

Health event streaming to CloudWatch Logs

Configure Elastic Beanstalk to stream environment health events to CloudWatch Logs. You can set the retention up to a maximum of ten years and configure Elastic Beanstalk to delete the logs when you terminate your environment.

Log streaming

standard CloudWatch charges apply.

- ☒ Enable

Retention

1

Lifecycle

Delete logs upon termination

Step 3: Configure Updates and Environment Variables

- Under Managed platform updates, I unchecked Activated. This gives greater control over when updates are applied to avoid disruption during development.

▼ Managed platform updates [Info](#)

Activate managed platform updates to apply platform updates automatically during a weekly maintenance window that you choose. Your application stays available during the update process.

Managed updates

- ☐ Enable

- Under Environment properties, I added:
 - Name: FRONTEND_URL (Value left blank for now). This environment variable will be used for CORS configuration, allowing secure communication between frontend and backend components.

Environment properties

Use environment properties as either plain text environment variables or as references to secret ARN values that are stored in AWS Secrets Manager or Systems Manager (SSM) Parameter Store. When using secrets, update your Elastic Beanstalk EC2 instance profile role with permissions to your secrets parameters. [Learn more](#)

Source	Name	Value	
Plain text ▼	FRONTEND_URL		Remove

[Add environment property](#)

Create and Troubleshoot the Environment

Objective: Deploy the Backend Environment and Resolve Common Issues

Completed the deployment process and troubleshoot potential issues that arose during Elastic Beanstalk environment creation.

Step 1: Created the Environment

- Reviewed all settings carefully before submission:
 - Verified application and environment configurations
 - Checked platform, database, and monitoring settings
 - Ensured all details aligned with my project requirements

Step 1: Configure environment

Environment information

Environment tier

Web server environment

Application name

backend-repairs

Environment name

Backend-repairs-env

Application code

backend.zip

Platform

arn:aws:elasticbeanstalk:us-east-1::platform/Node.js 24 running on 64bit Amazon Linux 2023/6.11.1

Step 2: Configure service access

Service access [Info](#)

Configure the service role and EC2 instance profile that Elastic Beanstalk uses to manage your environment. Choose a role that can securely log in to your EC2 instances.

Service role

arn:aws:iam::103549976993:role/aws-elasticbeanstalk-service-role

EC2 instance profile

aws-elasticbeanstalk-ec2-role

Step 3: Set up networking, database, and tags

Networking, database, and tags [Info](#)

Configure VPC settings, and subnets for your environment's EC2 instances and load balancer. Set up an Amazon RDS database that is integrated with your environment.

Database**Database availability**

false

Has coupled database

true

Database deletion policy

Delete

Database engine

mysql

Database engine version

8.4.8

Database instance class

db.t3.small

Database password

Database storage

20

Database username

ToDoA

Step 4: Configure instance traffic and scaling

[Edit](#)

Instance traffic and scaling [Info](#)

Customize the capacity and scaling for your environment's instances. Select security groups to control instance traffic. Configure the software that runs on your environment's instances by setting platform-specific options.

Instances

IMDSv1

Disabled

Capacity

Environment type

Single instance

Fleet composition

On-Demand instance

On-demand base

0

On-demand above base

0

Capacity rebalancing

Disabled

Scaling cooldown

360

Processor type

x86_64

Instance types

t3.micro,t3.small

AMI ID

ami-0a7c613066c3c3e8b

Step 5: Configure updates, monitoring, and logging

[Edit](#)

Updates, monitoring, and logging [Info](#)

Define when and how Elastic Beanstalk deploys changes to your environment. Manage your application's monitoring and logging settings, instances, and other environment resources.

Monitoring

System

enhanced

Cloudwatch custom metrics - instance

—

Cloudwatch custom metrics - environment

—

Log streaming

Enabled

Retention

1

Lifecycle

true

Updates

Managed updates

Disabled

Deployment batch size

100

Deployment batch size type

Percentage

Command timeout

600

Deployment policy

AllAtOnce

Health threshold

Ok

Ignore health check

false

Instance replacement

false

Platform software

- Submitted to create the application.
- The creation process took about 5-10 minutes.

Elastic Beanstalk is launching your environment. This will take a few minutes.

[×](#)

Backend-repairs-env [Info](#)

[Actions](#)[Upload and deploy](#)[Environment overview](#)[Platform](#)[Change version](#)

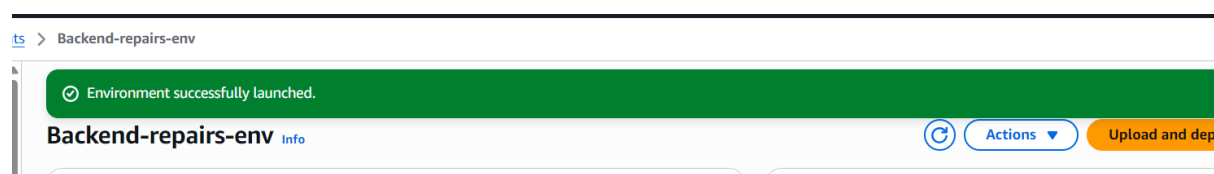
AWS Elastic Beanstalk automatically launched a CloudFormation stack to create the following required resources:

- **EC2 Instances:** Compute resources to host the application
- **RDS Database:** Managed MySQL database for the application
- **Security Groups:** Network access controls to manage inbound and outbound traffic
- **IAM Roles:** Necessary permissions for services to interact securely
- **CloudWatch Logs:** Monitoring and logging infrastructure
- **Elastic IP:** Static IP address for the application (if configured)

This automated resource creation simplifies infrastructure management and ensures consistent deployment. All resources are created and configured to work together seamlessly.

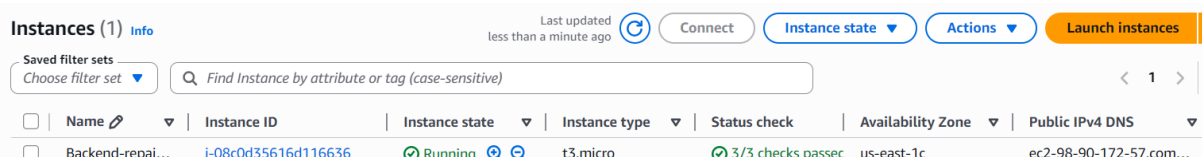
Verified Successful Deployment

1. I checked that the environment was created successfully.



2. I validated resource creation by navigating to the following AWS services:

- EC2
- RDS



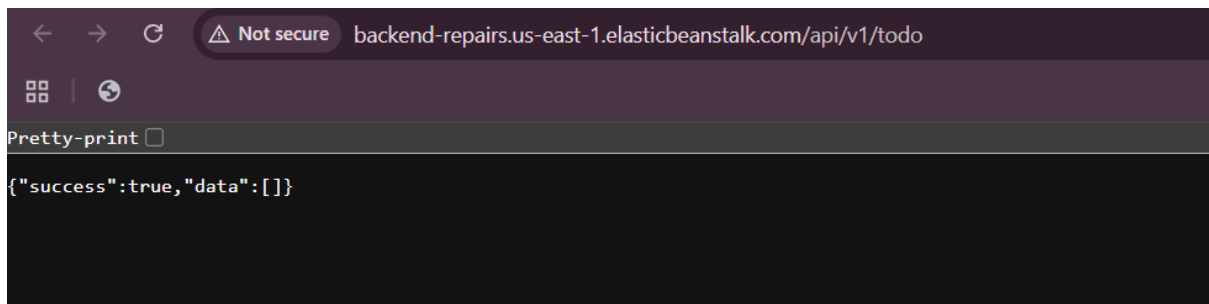
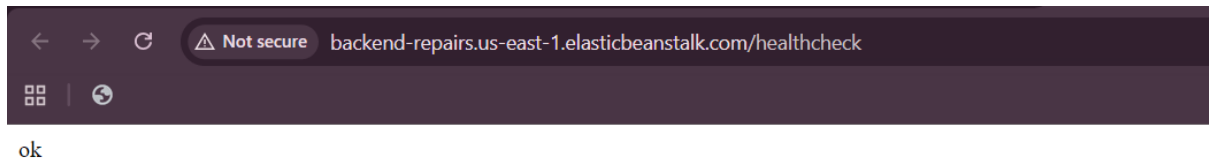
3. Application URL

- I noted the application URL upon completion.

- URL Format: <http://backend-repairs.us-east-1.elasticbeanstalk.com/>

4. Backend Functionality Test

- Tested the backend functionality by visiting these endpoints:
 - Health check: <http://backend-repairs.us-east-1.elasticbeanstalk.com/healthcheck>
 - API Endpoint: <http://backend-repairs.us-east-1.elasticbeanstalk.com/api/v1/todo>
- I verified that both the health check and API endpoint returned expected responses.



Upon seeing these responses, I had successfully deployed the backend of the application and proceeded to deploy the frontend.

Security Group Misconfiguration and CloudWatch Troubleshooting

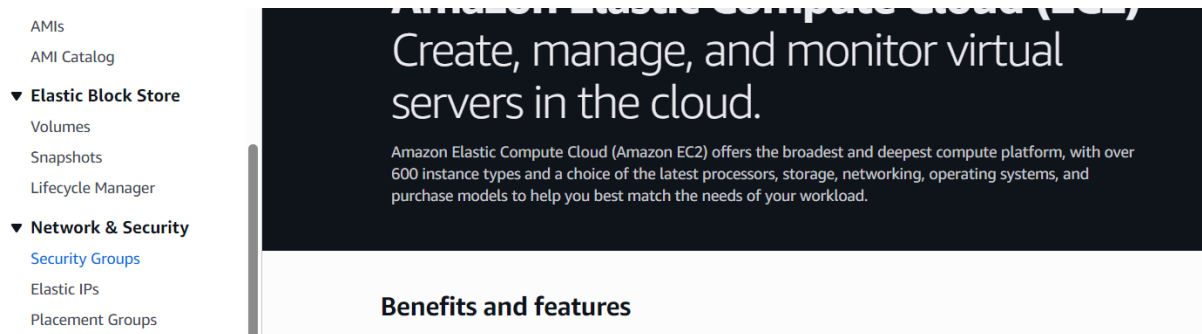
Creating and Resolving a Security Group Misconfiguration

Intentionally created a security group misconfiguration by removing HTTP access, observed the resulting application outage, used CloudWatch to diagnose the issue, and then resolved it. This simulates a common production scenario where security group changes can inadvertently block application access.

Identified My Elastic Beanstalk Security Group

1. Navigated to the EC2 Console:

- I searched for EC2 and selected it under Services
- I selected Security Groups in the left navigation panel



2. located the Elastic Beanstalk Security Group:

- Looked for security groups with names containing my environment name
- Found the group named awseb-e-tt25hceg8t-stack-AWSEBRDSDBSecurityGroup-aW1fLLIKTrJk
- Noted the Security Group ID ([sg-0e79818c0b9e52fdd](#))

Security Groups (1/6) [Info](#)

Find security groups by attribute or tag

Actions Export security groups to CSV Create security group

	Name	Security group ID	Security group name	VPC ID	Description
☐	-	sg-0fe454fb822c1df5f	default	vpc-00ecc19fd940b1efc	default VPC security group
☐	-	sg-020ae5a5873fe1999	launch-wizard-1	vpc-00ecc19fd940b1efc	launch-wizard-1
☒	Backend-repairs-env	sg-0e79818c0b9e52fdd	awseb-e-tt25hceg8t-stack-AWSEBRDSD...	vpc-00ecc19fd940b1efc	RDS DB VPC Security Group
☐	-	sg-07da54fa75ac6d620	launch-wizard-3	vpc-00ecc19fd940b1efc	launch-wizard-3
☐	-	sg-0889d9308de4e36bc	launch-wizard-2	vpc-00ecc19fd940b1efc	launch-wizard-2

Documented Original Configuration (Before Changes)

- Recorded the existing inbound rules:
 - Selected the security group
 - Went to the Inbound rules tab
 - Took screenshots and noted down the current rules

Inbound rules (1)

Search

Manage tags Edit inbound rules

	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
☐	-	sgr-080e4b3307ed43066	-	All TCP	TCP	0 - 65535	sg-00efd6174139f666e...	-

- Verified the presence of HTTP (port 80) and HTTPS (port 443) rules
- Confirmed both rules existed before making any changes

Introduced the Security Group Issue

- Modified the inbound rules:
 - I selected the security group
 - I clicked Edit inbound rules

Inbound rules (1)

Search

Manage tags Edit inbound rules

	Name	Security group rule ID	IP version	Type	Protocol	Port range	Source	Description
☐	-	sgr-080e4b3307ed43066	-	All TCP	TCP	0 - 65535	sg-00efd6174139f666e...	-

- Removed the HTTP (port 80) rule completely

Edit inbound rules [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules [Info](#)

This security group has no inbound rules.

[Add rule](#)

[Cancel](#)

[Preview changes](#)

[Save rules](#)

- Saved rules

2. Documented what I changed:

- Noted the exact rule that was removed
- Recorded the time when I made the change

Observed and Documented the Issue

1. Tested application access:

- Tried to access my application's URL
- Documented that the application became inaccessible
- Noted the error message in the browser — a connection timeout

← → ↻ 🏠 ⓘ about:blank

Gateway Timeout

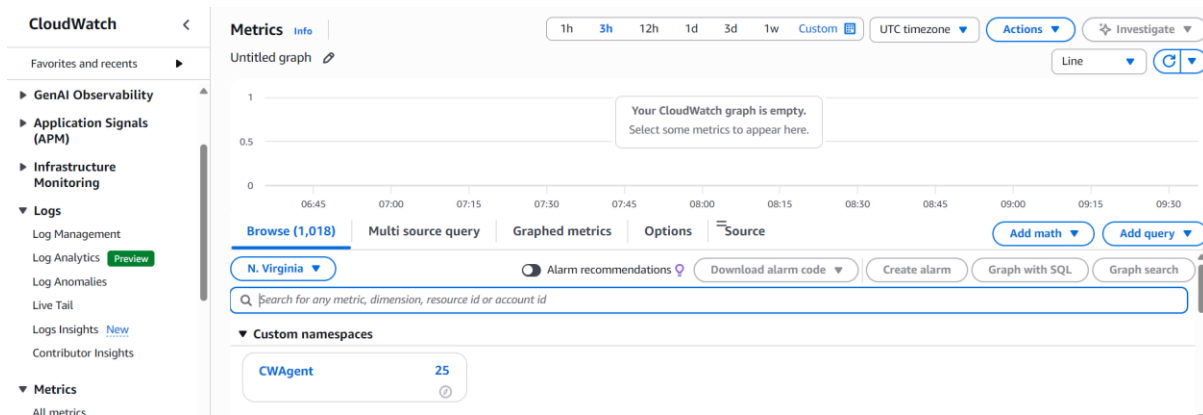
CloudWatch Troubleshooting Process

1. Navigated to CloudWatch:

- went to AWS Management Console
- searched for CloudWatch and selected it under Services

2. Checked relevant metrics:

- Went to Metrics → All metrics



- searched for my Elastic Beanstalk environment metrics
- looked specifically for:
 - RequestCount (dropped to zero or near zero)
 - EnvironmentHealth metric

	Instance name 7/7	VolumeId	InstanceId	Metric name	Alarms
☐	Backend-repairs-env	vol-0486907902a90...	i-08c0d35616d11...	VolumeAvgIOPS ⓘ	No alarms
☐	Backend-repairs-env	vol-0486907902a90...	i-08c0d35616d11...	VolumeAvgThroughput ⓘ	No alarms
☐	Backend-repairs-env	vol-0486907902a90...	i-08c0d35616d11...	VolumeAvgWriteLatency ⓘ	No alarms
☐	Backend-repairs-env	vol-0486907902a90...	i-08c0d35616d11...	VolumeThroughputExceededCheck ⓘ	No alarms

Diagnosed the Issue

- Navigated to CloudWatch Log Groups:
 - Went to AWS Management Console → CloudWatch → Log groups
 - Found my Elastic Beanstalk log group
 - Clicked on the log group and checked for logs

Timestamp	Message
	No older events at this moment. Retry
2026-06-04T08:54:49.000Z	{"id": "f915cd5d-206b-48b5-ab55-d4b938e27445", "version": "1.0", "type": "HealthTransition", "color": "Grey", "timestamp": "2026-06-04T08:54...
2026-06-04T08:54:59.000Z	{"id": "da0f2746-3dd6-47e6-8672-749aa6340987", "version": "1.0", "type": "HealthTransition", "color": "Grey", "timestamp": "2026-06-04T08:54...
2026-06-04T08:55:09.000Z	{"id": "0fa81356-206e-4712-83ff-2abc072eae4d", "version": "1.0", "type": "HealthTransition", "color": "Grey", "timestamp": "2026-06-04T08:55...
2026-06-04T08:55:19.000Z	{"id": "58834c82-8b60-45f6-a19e-fd19c784f0b2", "version": "1.0", "type": "HealthTransition", "color": "Grey", "timestamp": "2026-06-04T08:55...
2026-06-04T08:55:28.000Z	{"id": "bbf8b008-33ed-4802-9df1-a7270a09ed8e", "version": "1.0", "type": "HealthTransition", "color": "Grey", "timestamp": "2026-06-04T08:55...
2026-06-04T08:55:38.000Z	{"id": "bdd0183b-718e-4833-b3fb-01780f474399", "version": "1.0", "type": "HealthTransition", "color": "Grey", "timestamp": "2026-06-04T08:55...
2026-06-04T08:55:49.000Z	{"id": "14ccd918-1615-43cf-8cf6-2d9191a0f9e3", "version": "1.0", "type": "HealthTransition", "color": "Grey", "timestamp": "2026-06-04T08:55...
2026-06-04T08:55:58.000Z	{"id": "e78ee15c-62f5-4f10-8a5e-ddb186b0b1f", "version": "1.0", "type": "HealthTransition", "color": "Grey", "timestamp": "2026-06-04T08:55...

- Selected and analyzed relevant Log Streams:
 - Found streams containing access
 - Noticed that HTTP requests suddenly stopped appearing after modifying the security group
 - Checked health check log streams (/var/log/eb-health.log) for failure patterns
- Documented my findings:
 - Noted the exact time when logs stopped showing incoming requests
 - Compared this with the time I modified the security group
 - The correlation between these times confirmed the security group as the cause of the issue
 - Documented that no new entries appeared in access logs after the security group change

Resolved the Issue

1. Restored the proper security group rules:
 - Went to EC2 Console → Security Groups → Backend-todo-repairs
 - Added back the HTTP rule:
 - Type: HTTP
 - Protocol: TCP
 - Port Range: 80
 - Source: 0.0.0.0/0

EC2 > Security Groups > sg-00efd6174139f666e - awseb-e-tt25hceg8t-stack-AWSEBSecurityGroup-og9f46EMhQ8K > Edit inbound rules

Edit inbound rules [Info](#)
Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules Info	Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info
sg-049db1bc78c7bebc1	HTTP	TCP	80	Custom	0.0.0.0/0

[Add rule](#) [Delete](#)

- Saved the rules

2. Verified the resolution:

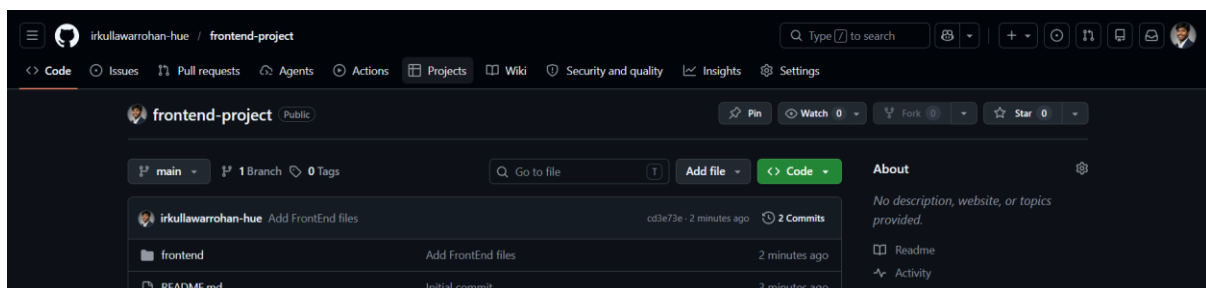
- Tested application access again
- Confirmed the application was accessible
- Monitored CloudWatch metrics to see traffic resuming

Deploying the Frontend on AWS Amplify

When developing a web application with separate frontend and backend components, a reliable way to host the frontend, connect it to the backend API, and ensure continuous deployment of updates was needed. AWS Amplify solved this challenge by providing a complete solution for deploying and hosting web applications with continuous integration and delivery capabilities.

Frontend Repository

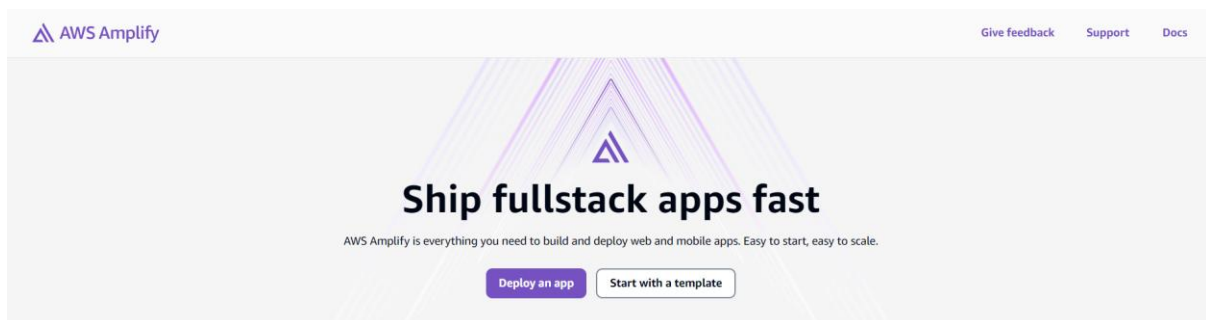
- Navigated to the GitHub repository



Deployed via AWS Amplify

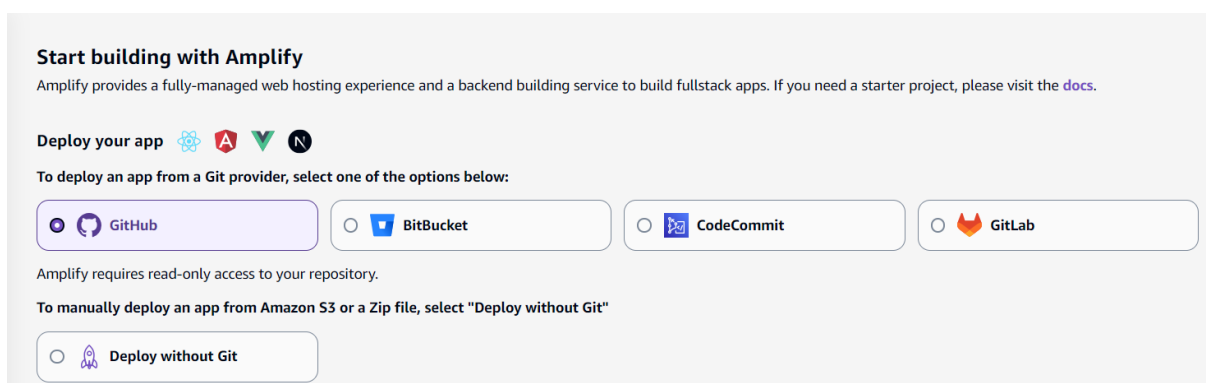
1. Access AWS Amplify

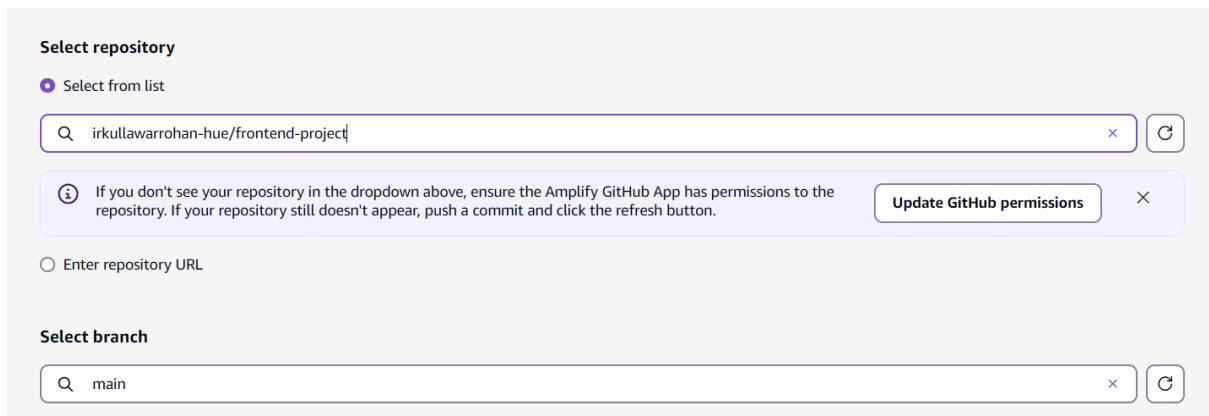
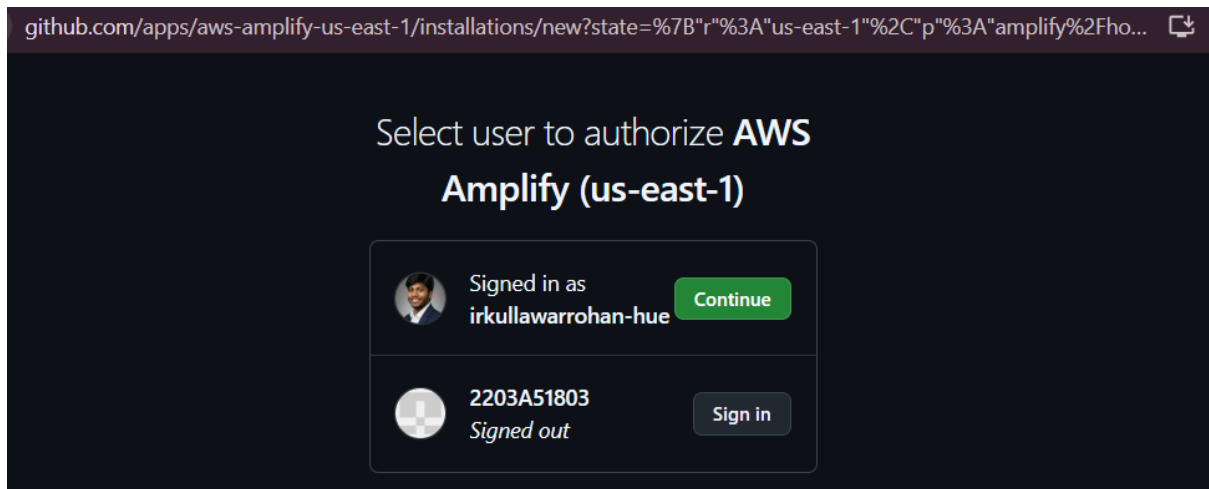
- Went back to the AWS Console and searched for Amplify
- Selected Deploy an app
- AWS Amplify simplifies the deployment process by automatically building, testing, and deploying the application whenever code changes are pushed to the repository



Connected to GitHub

- Chose Source code provider as GitHub, then clicked Next
- This integration enables automated deployments directly from the GitHub repository, creating a seamless CI/CD pipeline
- Authorized AWS Amplify to access the GitHub account
- This one-time authorization allows Amplify to retrieve code and set up webhook notifications for future updates





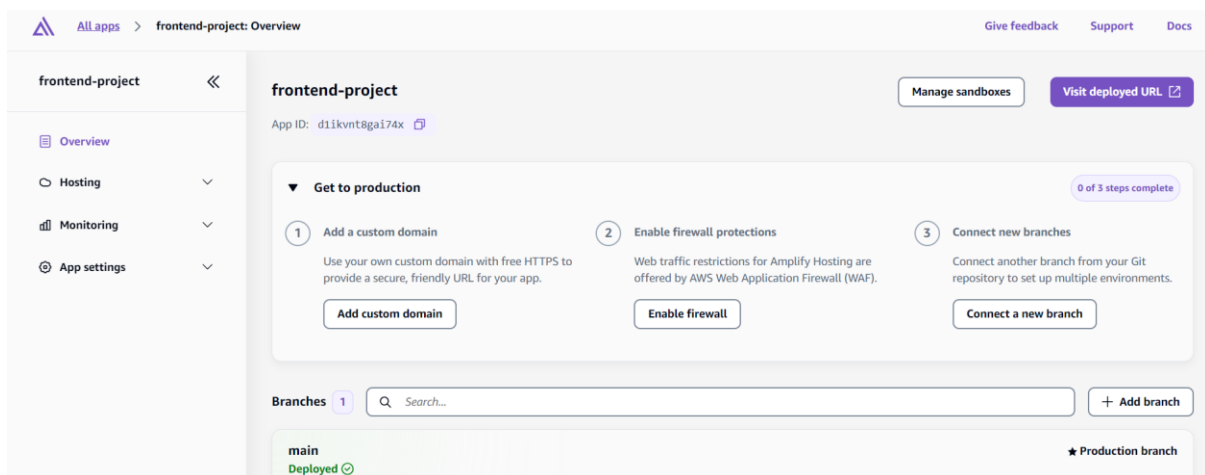
After deploying the frontend application with AWS Amplify, the application appeared to load but didn't connect to the backend API. This happened because the frontend didn't know where to find the backend, and the backend was not configured to accept requests from the new frontend domain.

Setting Up Environment Variables and Testing

Add Environment Variables

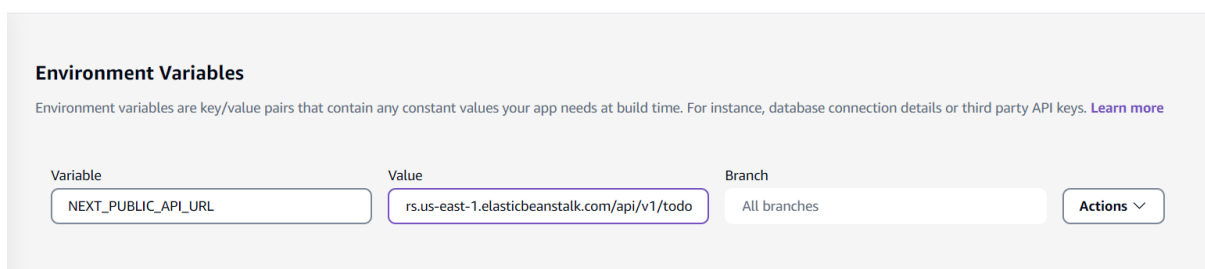
1. Access Environment Variables section

- While the app was deploying, navigated to Environment variables in the sidebar
- Environment variables allow configuration of the application without modifying code, making it easier to connect to different backends across environments



Configure API URL

- Clicked on Manage Variables
- Added a new variable:
 - Variable: NEXT_PUBLIC_API_URL
 - Value: Elastic Beanstalk URL followed by /api/v1/todo



Deployments

Deployment 2
Deploying ↗

Download Cancel build

Started at 6/4/2026, 3:46 PM Build duration 19 seconds Domain <https://main.d1ikvnt8gai74x.amplifyapp.com> Repository frontend-project:main Last commit Auto-build

Build Deploying ↗

Deploy Deploy pending ⌚

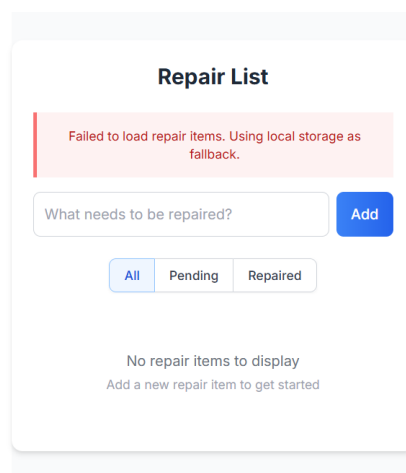
Deployment history 2

	Name	Status	Build duration	Commit message	Started at
☐	Deployment 2	Deployed ✓	53 seconds	Auto-build	6/4/2026, 3:46 PM
☒	Deployment 1	Deployed ✓	53 seconds	Auto-build	6/4/2026, 3:39 PM

Verified Deployment

1. Access the application

- Once deployment was complete, opened the Amplify app URL
- Example URL: <https://main.d2whyj8eoi6nra.amplifyapp.com/>
- The Repair application's frontend was visible
- At this stage, "Failed to load todos" appeared because the backend connection still needed to be configured properly



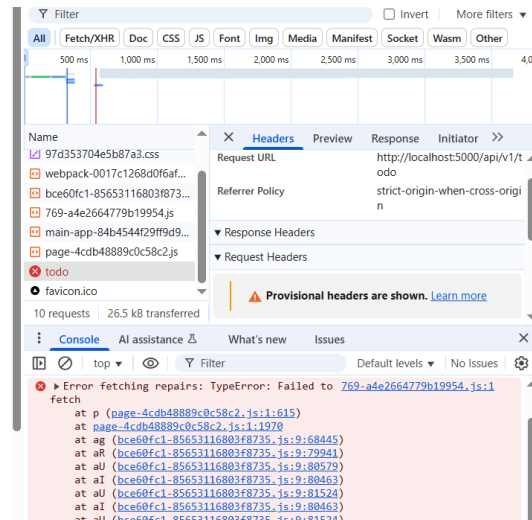
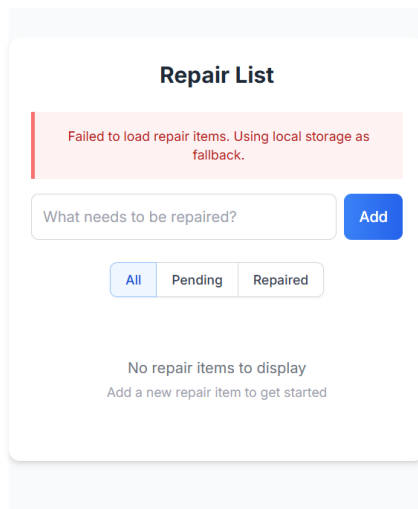
Tested functionality

- Tried adding a new repair task
- The repair task appeared to be added but was only stored in the browser's local storage, not in the RDS database

The screenshot shows a web interface titled "Repair List". At the top, there is a red error message: "Failed to add repair item. Added to local storage only." Below this is a form with a text input field containing the placeholder text "What needs to be repaired?" and a blue "Add" button. Under the input field are three filter buttons: "All" (highlighted in light blue), "Pending", and "Repaired". Below the filters is a list of repair items. The first item is "Fix Tap", which has a checkbox on the left and a trash icon on the right. At the bottom of the list, it says "0 of 1 repaired".

Verify connection status

- On the application page: Right clicked → Inspect → went to the Networks tab
- Clicked Ctrl+R to reload
- Noticed there were no requests going to the Elastic Beanstalk backend



Understanding the Connection Issue

The frontend and backend weren't properly connected yet because:

- The frontend needed to know the backend URL (which was configured)
- The backend needed to allow requests from the frontend domain (CORS configuration)
- The environment variables needed to be properly accessed in the application code

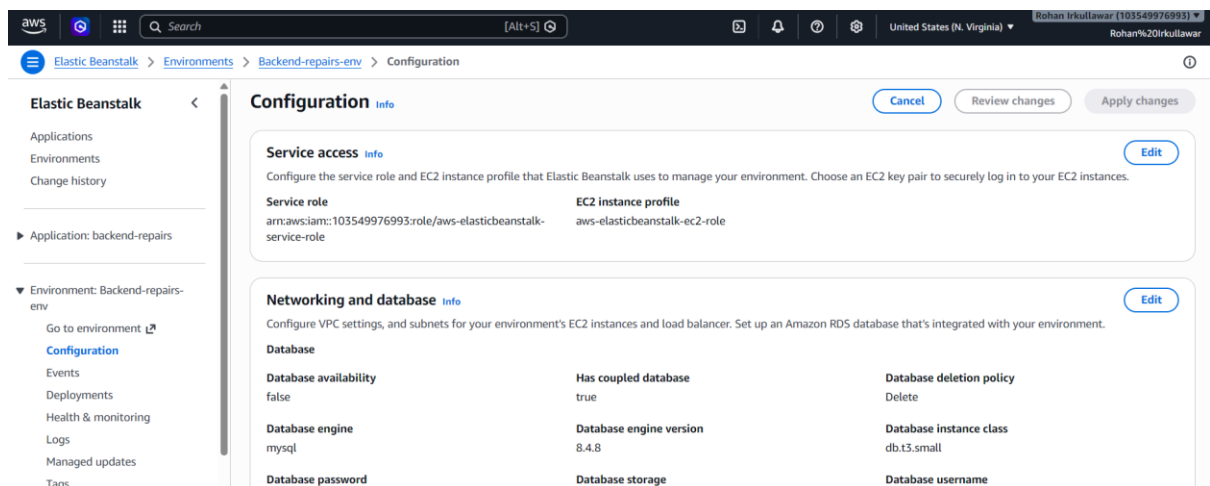
Connecting the Frontend to Backend

The Challenge: Secure Cross-Origin Communication

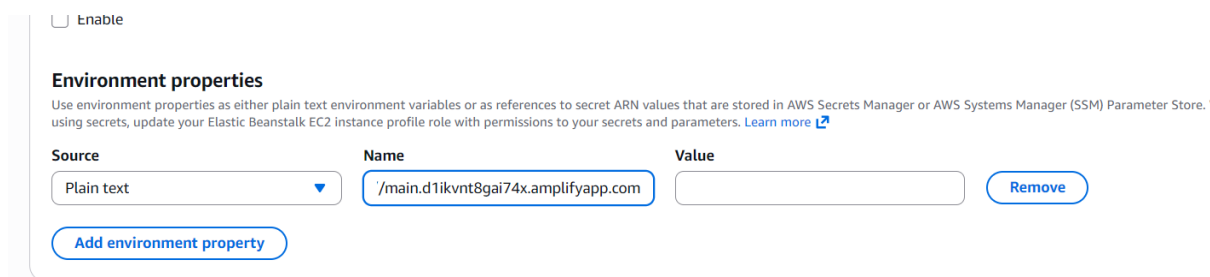
When hosting the frontend and backend on different domains (Amplify and Elastic Beanstalk), browsers block communication between them as a security measure. This challenge involved configuring CORS (Cross-Origin Resource Sharing) properly to allow secure communication while maintaining security standards.

Steps to Update CORS Settings

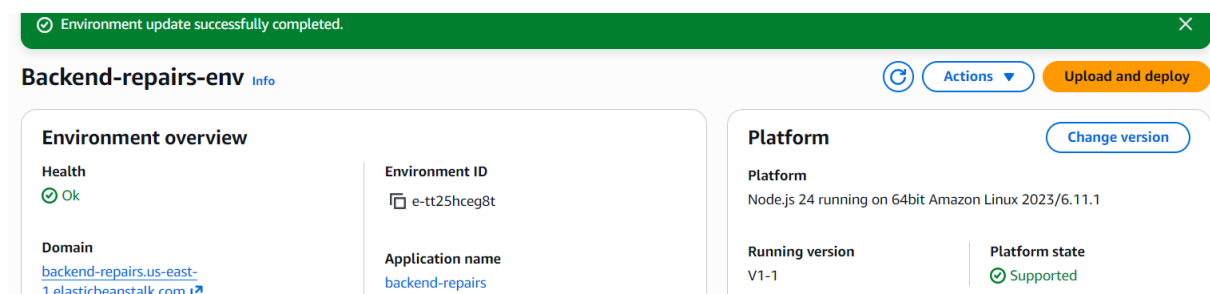
1. Navigated to Elastic Beanstalk in the AWS Console
2. Selected the backend-repairs application
3. Went to Configuration → Updates, monitoring, and logging → clicked Edit



4. In Environment properties:
 - Located the FRONTEND_URL property
 - Updated it with the Amplify URL



5. Clicked Apply
6. Allowed time for the environment to update completely



Testing the Application

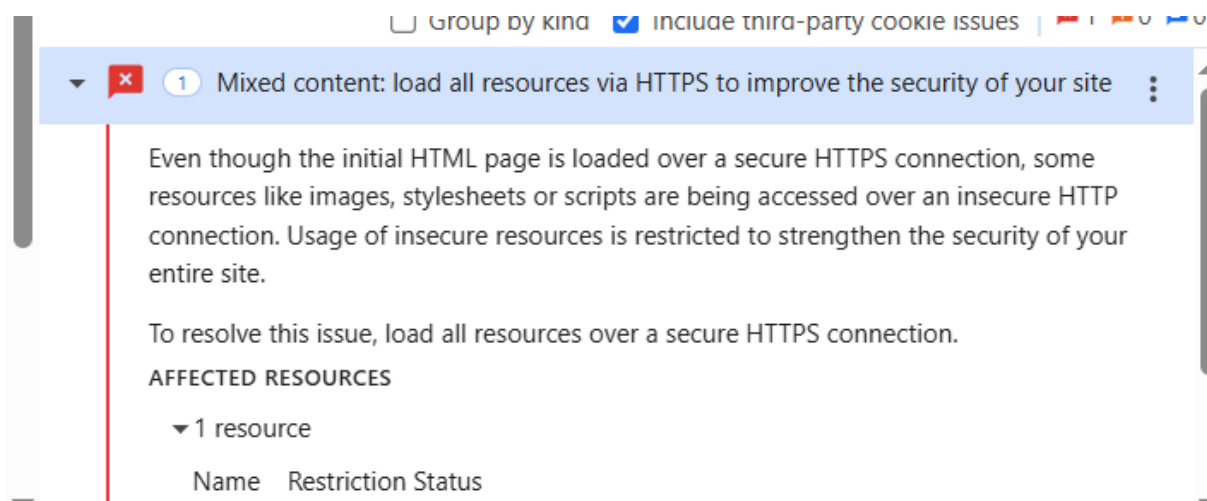
Testing confirms that the frontend and backend are properly connected and communicating. This verification step helps identify and resolve any configuration issues before making the application available to users.

Steps to Test

1. Opened the Amplify application URL in a browser
2. Verified that the application loads and functions as expected
3. When issues occurred, checked browser console errors:
 - Right-clicked on the page → Inspect → Console
 - Reviewed any error messages

The Problem: Mixed Content Security Issues

After connecting the frontend and backend, mixed content errors were encountered. This occurred because the frontend was served over secure HTTPS while the backend used non-secure HTTP. Modern browsers block these mixed-content requests for security reasons.



Troubleshooting Mixed Content Issues

1. Went to the Network tab in the browser's developer tools
2. Refreshed the page
3. Analyzed the requests:
 - Noticed that requests to `backend-todo.us-east-1.elasticbeanstalk.com/api/v1/todo` were blocked
 - This occurred because the frontend was served over HTTPS while the backend used HTTP
 - Modern browsers block these mixed-content requests for security reasons

Securing the Backend with CloudFront

The Problem: The frontend (HTTPS) could not safely communicate with the backend (HTTP) due to browser security restrictions. When users attempted to access backend resources from the secure frontend, browsers blocked these requests as mixed content, preventing the application from functioning correctly.

Using CloudFront as a Secure Proxy

After encountering the mixed content issue, AWS CloudFront was used as a reverse proxy for the Elastic Beanstalk backend. This allowed HTTPS communication to be enforced without making complex modifications to the backend code or configuration.

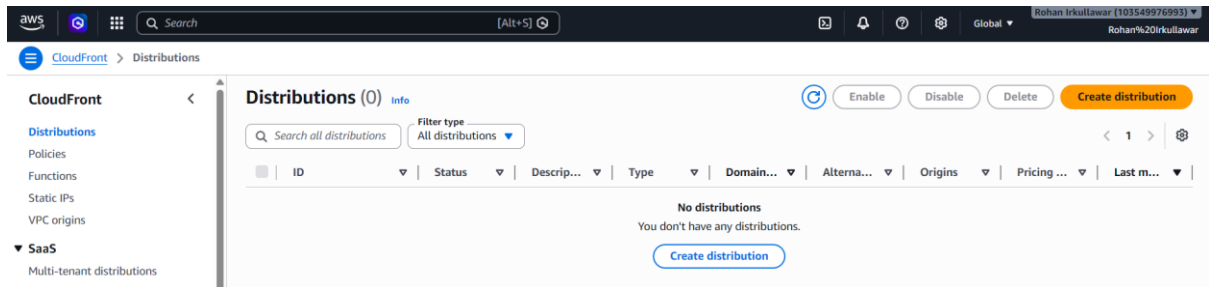
Benefits of Using CloudFront

- Enables HTTPS for the backend without direct Elastic Beanstalk modifications
- Remains within AWS Free Tier limits
- Adds a content delivery network (CDN) layer that can improve performance

- Provides additional security through AWS's edge locations

Step-by-Step CloudFront Configuration

1. Navigated to CloudFront in the AWS Console
2. Clicked Create distribution



3. Provided distribution name as Repairs-app

Distribution options [Info](#)

Distribution name
Name will be stored as a tag on the resource. You can add a name, or more tags, later.

Repairs-app

☒ **Single website or app**
Choose if each website or application will have a unique configuration.

☐ **Multi-tenant architecture - New**
Choose when you have multiple domains that need to share configurations. This is a common architecture for SaaS providers.

4. Selected origin type as Other, Origin as backend-repairs.us-east-1.elasticbeanstalk.com

Origin type
Your origin is where your content (such as a website or app) lives. CloudFront works with AWS-based origins and origins hosted on other cloud providers.

☐ **Amazon S3**
Deliver static assets like files and images, statically generated websites or single page applications (SPA).

☐ **Elastic Load Balancer**
Deliver applications hosted behind ELB such as dynamic websites, web services, and APIs.

☐ **API Gateway**
Deliver API endpoints for REST APIs hosted on API Gateway.

☐ **Elemental MediaPackage**
Deliver end-to-end live events or video on demand (VOD).

☐ **VPC origin**
Deliver applications and content hosted within private VPCs, such as EC2 instances and Application Load Balancers.

☒ **Other**
Refer to any AWS or non-AWS origin through its publicly resolvable URL.

Origin

Custom origin
Choose an AWS origin, or enter your origin's domain name. [Learn more](#)

backend-repairs.us-east-1.elasticbeanstalk.com

Origin path - optional
The origin path is the path to your content on the origin. For example, /static/

5. Under Settings → Origin settings → selected Customize origin settings
6. Set Protocol as HTTP Only

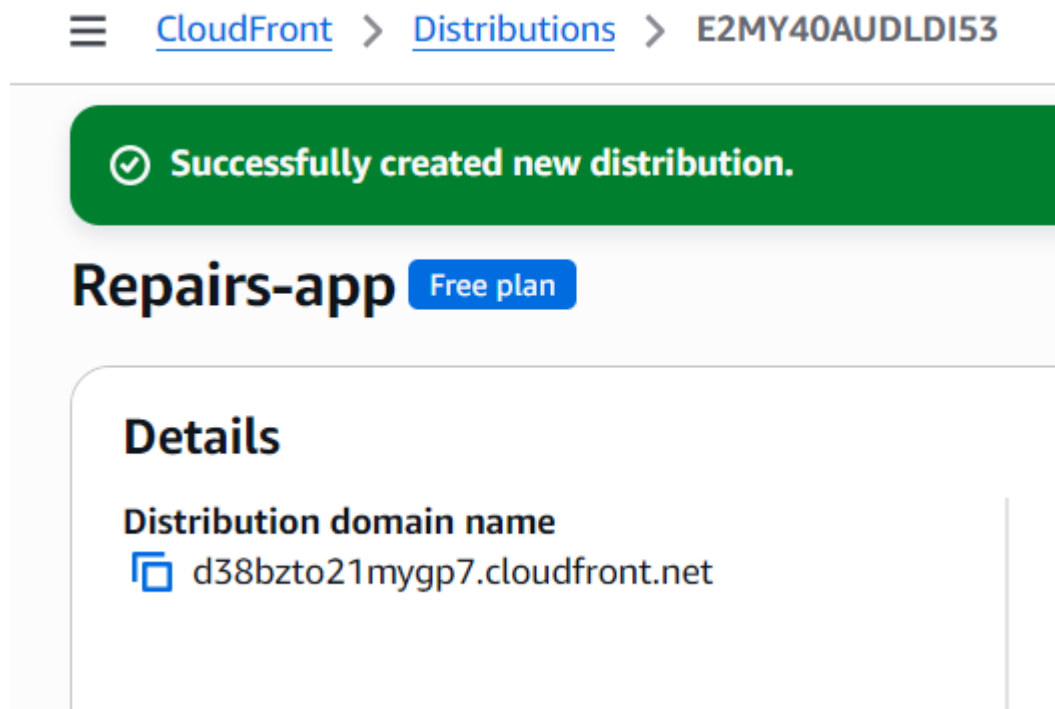
7. Left other settings at defaults and created the distribution

Updating Frontend to Use Secure CloudFront Endpoint

After securing the backend with CloudFront, the frontend application was pointed to the new secure endpoint. This ensured all communication used HTTPS throughout the entire application stack, eliminating mixed content security issues.

Steps to Update the Frontend

1. Once the CloudFront distribution status showed Deployed, copied the distribution domain name



The screenshot shows the AWS CloudFront console. At the top, there's a breadcrumb trail: CloudFront > Distributions > E2MY40AUDLDI53. Below this, a green banner with a checkmark icon says "Successfully created new distribution." Underneath, the text "Repairs-app" is followed by a blue button labeled "Free plan". A section titled "Details" contains the label "Distribution domain name" and a CloudFront icon followed by the domain "d38bzto21mygp7.cloudfront.net".

2. Updated the Amplify environment variable:

- Variable name: NEXT_PUBLIC_API_URL
- New value: <https://d1234abcdef.cloudfront.net/api/v1/todo>

Environment Variables

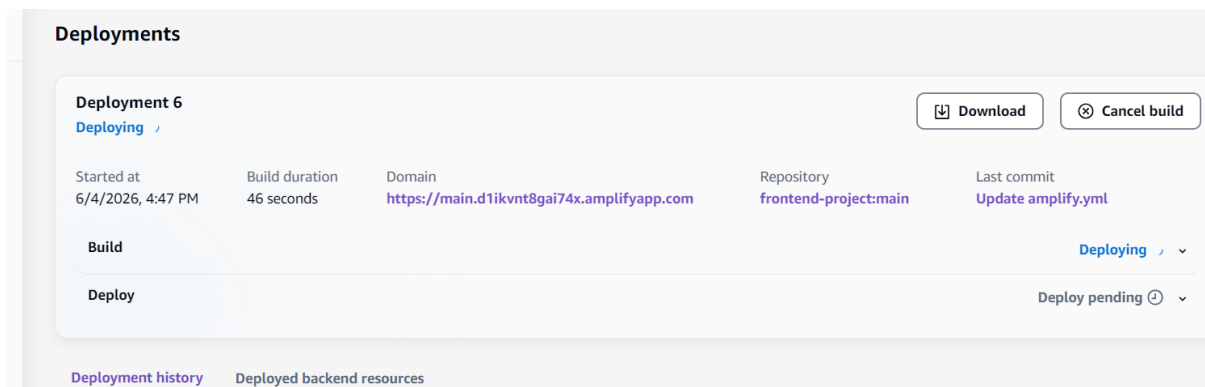
Environment variables are key/value pairs that contain any constant values your app needs at build time. For instance, database connection details or third party API keys. [Learn more](#)

Variable	Value	Branch	Actions
NEXT_PUBLIC_API_URL	https://d38bzto21mygp7.cloudfront.net/api/v1/todo	All branches	

Redeploying the Frontend

After updating the API endpoint in the environment variables, the frontend application was redeployed to incorporate these changes. This ensured the application used the new secure CloudFront URL for all API requests.

1. Navigated to the Overview section in AWS Amplify
2. Selected the application branch (main)
3. Clicked on Redeploy this Version
4. Waited for the redeployment process to complete (typically a few minutes)



Common Issue: CORS Configuration Mismatch

After redeployment, the following error appeared in the console:

The 'Access-Control-Allow-Origin' header has a value <https://main.d3gfrim0yehrbd.amplifyapp.com/> that is not equal to the supplied origin.

Application error: a client-side exception has occurred (see the browser console for more information).

```
https://d5jmh0js0tku9.cloudfront.net/api/v1/todo' from origin '
https://main.d3gfrim9yehrbd.amplifyapp.com' has been blocked by CORS policy: The
'Access-Control-Allow-Origin' header has a value '
https://main.d3gfrim9yehrbd.amplifyapp.com/' that is not equal to the supplied origi
Have the server send the header with a valid value, or, if an opaque response serv
your needs, set the request's mode to 'no-cors' to fetch the resource with CORS
disabled.

Failed to load resource: net::ERR_FAILED d5jmh0js0tku9.cloudfront.net/api/v1/todo:1
Error fetching todos: TypeError: Failed to fetch hook.js:1
    at u (page-5733e2dd2a5846e9.js:1:401)
    at page-5733e2dd2a5846e9.js:1:1364
    at ag (bce60fc1-85653116803f8735.js:9:68445)
    at ar (bce60fc1-85653116803f8735.js:9:79941)
    at au (bce60fc1-85653116803f8735.js:9:80579)
    at ai (bce60fc1-85653116803f8735.js:9:80463)
    at au (bce60fc1-85653116803f8735.js:9:81524)
    at ai (bce60fc1-85653116803f8735.js:9:80463)
    at au (bce60fc1-85653116803f8735.js:9:81524)
    at ai (bce60fc1-85653116803f8735.js:9:80463)

Error: Minified React error #31; visit hook.js:1
https://reactjs.org/docs/error-decoder.html?invariant=31&args[]=object%20with%20ke
for the full message or use the non-minified dev environment for full errors and
additional helpful warnings.
    at nz (bce60fc1-85653116803f8735.js:9:29494)
    at s (bce60fc1-85653116803f8735.js:9:35023)
    at u (bce60fc1-85653116803f8735.js:9:35207)
    at ls (bce60fc1-85653116803f8735.js:9:52286)
    at iv (bce60fc1-85653116803f8735.js:9:115646)
    at or (bce60fc1-85653116803f8735.js:9:90060)
    at bce60fc1-85653116803f8735.js:9:89909
```

This error occurred because the CORS configuration in the backend had a trailing slash (/) at the end of the URL, which didn't match exactly with the origin of the requests. Even small differences like this can cause CORS to fail.

Troubleshooting CORS Issues with CloudWatch Logs

CORS Configuration Mismatch

After redeploying the application with the new CloudFront endpoint, the Todo app failed to load data. The issue was a CORS configuration mismatch between the frontend origin and the allowed origins in the backend.

Used CloudWatch Logs to Diagnose CORS Issues

Access CloudWatch Logs

1. Went to the AWS Management Console
2. Searched for and selected CloudWatch
3. In the left sidebar, clicked on Log groups
4. Found the Elastic Beanstalk log group:

- Looked for: `/aws/elasticbeanstalk/Backend-repairs-env/var/log/web.stdout.log`
- Clicked on the log group name to open it

Examine Recent Log Streams

1. A list of log streams organized by date and time was visible
2. Clicked on the most recent log stream
3. Looked for CORS-related error messages — noticed messages similar to: 100.0 % of the requests are erroring with HTTP 4xx

Fix CORS Issues

1. Went to the Elastic Beanstalk Console
2. Navigated to Environment → Backend-todo-env → Configuration
3. Found the Environment Variables section
4. Edited the value of `FRONTEND_URL` to remove the trailing slash:
 - Incorrect: <https://main.d3gfrim0yehrbd.amplifyapp.com/>
 - Correct: <https://main.d3gfrim0yehrbd.amplifyapp.com>
5. Clicked Apply
6. Waited for the environment to update

Environment properties

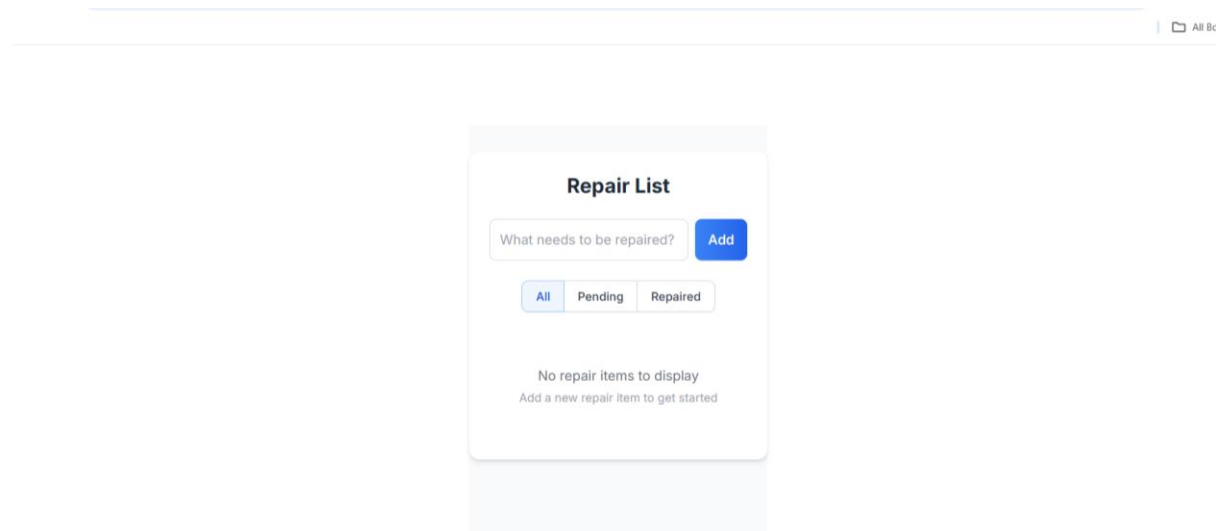
Use environment properties as either plain text environment variables or as references to secret ARN values that are stored in AWS Secrets Manager or AWS Systems Manager (SSM) Parameter Store. For more information, see [Using secrets, update your Elastic Beanstalk EC2 instance profile role with permissions to your secrets and parameters](#). [Learn more](#)

Source	Name	Value	
Plain text ▼	FRONTEND_URL	https://main.d1kvnt8gai74x.amplifyapp.com	Remove

[Add environment property](#)

Verified the Solution

1. Accessed the Amplify application URL
2. Confirmed that the Repair application loaded properly



3. Tested the functionality by adding a few Repair tasks
4. Verified the network requests:
 - Right-clicked → Inspect → Network Tab
 - Refreshed the page
 - Checked the todo requests
 - Confirmed they were going to the CloudFront URL:
<https://d38bzto21mygp7.cloudfront.net/api/v1/todo>

1. Optionally verified that data was being stored correctly in the backend database

Step 5: Verified the Fix in CloudWatch Logs

1. Returned to the CloudWatch log group
2. Opened the newest log stream (created after the environment update)
3. Used the same filter as before

4. Looked for successful CORS responses

Final Verification

1. Accessed the Amplify application URL
2. Confirmed that the Repairs application loaded properly
3. Checked CloudWatch logs for successful API requests from the frontend through CloudFront

Final Result:

📁 All Bc

